CS6008 Network Security, Assignment 2: DNS Security

Report

Team: Shiv Narang & Sabil

October 2026

Overview

We build a small DNS hierarchy in Docker across three isolated subnets that are joined only by two no-NAT routers. First we run plain DNS, then attack it from an on-path position, then add defences one at a time (DNSSEC, and finally an encrypted last mile) and see which attack each one actually stops.

Addresses. client `172.16.1.10`, webserver `172.16.1.20`, rogue-web `172.16.1.30`; resolver `172.16.2.10`, attacker `172.16.2.100` (phases 2 to 5) or `172.16.1.100` (phases 6 to 7); root-ns `172.16.3.10`, tld-ns `172.16.3.20`, domain-ns `172.16.3.30`; edge-router `172.16.1.2`/`172.16.2.2`, core-router `172.16.2.3`/`172.16.3.3`.

Shared code (`common/`). `base/` has the Phase 1 hierarchy and the BIND 9.20 image, `signing/` does the DNSSEC signing, and `interceptor/` is the C/libpcap attack tool. Every `phaseN/` folder is self-contained and reuses these.

We ran everything on Colima (Docker 29, Ubuntu 24.04 VM, Apple Silicon). Each result below comes from the `scripts/verify.sh` or `scripts/attack.sh` in that phase's folder.

Phase 1: Building the DNS Hierarchy

Design choices

Isolation comes from the topology, not from firewall rules. Each subnet is a Docker bridge with `internal: true`, so there is no NAT and no Internet. A packet only crosses subnets through a router that sits on both bridges, and the forwarding happens inside that router container, so we never have to fight Docker's own bridge isolation. Nothing else forwards.

Each node installs its own static routes from `STATIC_ROUTES` in a small entrypoint, which only needs `NET_ADMIN`: client reaches the resolver net via edge-router, the resolver reaches the auth net via core-router and the client net via edge-router, and each nameserver reaches the resolver net via core-router. The routers need no extra routes since both of their subnets are directly connected.

Addresses stay real end to end: root hints and glue point at the nameservers' `172.16.3.x` addresses. The resolver is recursive with DNSSEC validation off.

Report answers

Resolution, fetch, and the isolation check:

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client dig +short www.example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i welcome
  <h1>Welcome to example.com</h1>
$ docker exec client dig +time=2 +tries=1 @172.16.3.10 example.com
```

```
;; no servers could be reached        # auth subnet is not directly reachable
```

The referral chain, from a capture on core-router. One `example.com/A` resolution crosses it as these exchanges (counted by destination):

```
16 queries -> 172.16.3.10 (root-ns)    # . and com, plus resolving the NS names
 2 queries -> 172.16.3.20 (tld-ns)     # referral to example.com + glue
 2 queries -> 172.16.3.30 (domain-ns)  # final A answer 172.16.1.20
```

Everything on the wire is cleartext. You can read the QNAME being asked, the 16-bit query ID and UDP source port that tie a reply to its query, the record type, and the answers (the referral NS names with their glue, then `example.com A 172.16.1.20`). The root is hit many times only because our nameserver names live in their own pseudo-TLDs; resolution still finishes from the glue.

How the resolver's query reaches the root:

```
$ docker exec resolver ip route get 172.16.3.10
172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10
```

The resolver has no leg on the auth subnet, so it uses the static route `172.16.3.0/24 via 172.16.2.3` and hands the packet to core-router, which forwards it out of its `172.16.3.3` interface to root-ns.

Because the whole exchange is unauthenticated cleartext, any node on the path can read a query and forge a reply by copying its QNAME, ID, and ports. That is exactly Phase 2.

Phase 2: On-Path Response Forgery

Design choices

The diversion is a single route on the router. We add `ip route replace 172.16.2.10/32 via 172.16.2.100` on core-router, so the resolver's inbound responses go to the attacker first. The resolver and nameservers keep their Phase 1 config.

The attacker does not forward. It has `NET_RAW` (plus `NET_ADMIN`) and runs the C tool in `common/interceptor/dns_intercept.c` Since the kernel would drop the diverted frames (wrong destination IP, and `ip_forward=0`), we use `pcap_open_live` to capture and `pcap_inject` to relay them, which is the capture-and-inject path the brief asks for.

The forge is selective. A BPF filter picks only the diverted DNS responses (`ether dst <us> and udp and src port 53 and dst host 172.16.2.10`), and never our own injected frames, so there is no loop. We rewrite the A record only when the question is exactly `example.com` with type A. IP and UDP checksums are recomputed and the packet length does not change.

Evidence

```
# relay mode (forging off): transparent
example.com     -> 172.16.1.20   ("Welcome to example.com")

# forge mode:
example.com     -> 172.16.1.30   (rogue "HACKED" page)
www.example.com -> 172.16.1.20   (left alone)
```

Report answers

Plain DNS has no origin authentication. A resolver accepts any reply that matches the query's QNAME, type, 16-bit ID, and UDP port. An off-path spoofer has to guess the ID and port and win a race against the real server. Our on-path attacker just reads them from the genuine packet and rewrites it, so there is nothing to guess and no race, and the attack is deterministic.

Non-target names are safe because the tool only rewrites answers when the QNAME is exactly `example.com` and the type is A. `www.example.com` is a different QNAME, so it falls through the relay path and is injected byte for byte, which is why it still returns `172.16.1.20`.

Phase 3: DNSSEC Chain of Trust

Design choices

We sign with ECDSAP256SHA256 (algorithm 13). The keys and signatures are small and validation is fast, and it is the modern default. The trade-off is a shorter track record than RSA and slightly less universal support than it is new (Ed25519 is even newer), but it keeps the signed zones compact and is widely supported.

The chain is built child first (`common/signing/sign-all.sh`): sign `example.com`, put its KSK's DS into `com`, sign `com`, put its DS into the root, then sign the root. The resolver anchors on the root KSK only (`trust-anchor.conf`) and turns on `dnssec-validation yes`. Root hints are unchanged from Phase 1. The attacker and its Phase 2 binary are untouched.

No private key is in the repository. Keys and signed zones are build artifacts that the signer regenerates, and they are git-ignored.

Verify (healthy signed hierarchy)

```
$ dig +dnssec example.com   ->  status: NOERROR; flags: qr rd ra ad ; + RRSIG
$ delv -a trust-anchor.conf @172.16.2.10 example.com
; fully validated
example.com.  3582  IN  A  172.16.1.20
```

Attack (Phase 2 forgery re-run, unchanged)

```
$ docker exec client dig example.com
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL        # forged answer rejected
$ docker exec client dig +short www.example.com
172.16.1.20                                           # untargeted name still validates
# resolver log:
validating example.com/A: no valid signature found
RRSIG failed to verify resolving 'example.com/A/IN': 172.16.3.30#53
# client fetch: no page, resolution failed, attack defeated
```

Report answers

The attacker injects the same forged A, but it cannot produce a matching RRSIG because it has no zone private key. The validating resolver walks the chain (root KSK, DS(com), com KSK, DS(example.com), example.com ZSK) and checks the signature over the A RRset. It finds no valid signature, so it cannot call the name secure and returns SERVFAIL instead of the poison. In Phase 2 the same packet was accepted because nothing was checked.

On the wire the exchange is still cleartext, so the on-path attacker can read the QNAME, the IDs, and the answers just like in Phase 1, now with the RRSIG, DNSKEY, and DS records visible too. So DNSSEC gives origin authentication and integrity of the DNS data, but not confidentiality, and it does not protect the last hop to a stub that does not validate. That last hop is Phase 6.

Private keys do not belong in the repo. We commit only the signing scripts and the public trust anchor; the KSK and ZSK private keys are regenerated each run.

Phase 7: Encrypting the Last Mile (DoT / DoH), deploy half

Our split: this half covers the deployment, the encrypted resolution, and the no-cleartext check. The three attack attempts in 7.3 build on the Phase 6 attacker and are Sabil's half.

Design choices

We stand up a private CA (`pki/gen-ca.sh`): an RSA-2048 root CA issues the resolver a certificate whose CN and SAN are `resolver.cs6008.local`, with a DNS SAN only and no IP SAN. The CA goes into the client's system trust store with `update-ca-certificates`, so there is no `-k` or insecure bypass.

The client reaches the resolver by name, never by IP. It has `extra_hosts: resolver.cs6008.local -> 172.16.2.10` and connects to that name, so the certificate is checked against the hostname. On the resolver we add `tls local-tls { cert-file; key-file; }` with `listen-on port 853 tls` for DoT and `listen-on port 443 tls ... http` for DoH. Plain port 53 stays up so the later downgrade attempt has a fallback to target. The Phase 3 DNSSEC validation still runs upstream.

Verify

```
# DoT:
;; TLS session (TLS1.3)-(ECDHE-X25519)-(RSA-PSS-RSAE-SHA256)-(AES-256-GCM)
;; status: NOERROR; Flags: qr rd ra ad ;  example.com. IN A 172.16.1.20
;; From 172.16.2.10@853(TLS)
# DoH:
;; HTTP session (HTTP/2-POST)-(resolver.cs6008.local/dns-query)-(status: 200)
;; From 172.16.2.10@443(HTTPS)
# validation really is enforced:
+tls-hostname=wrong.name.local  -> WARNING: TLS, handshake failed (certificate error)
missing CA file                 -> ERROR: failed to initialize crypto context
# no cleartext (edge-router capture during a DoT lookup):
cleartext :53 packets = 0        encrypted :853 packets = 31
```

Report answers (our half)

We deployed both DoT and DoH and lead with DoT. DoT on its own port 853 is simple and easy to audit, but just as easy to spot and block. DoH on 443 blends in with normal HTTPS and is much harder to block, but it is harder to audit and heavier because of the HTTP/2 framing. It is a visibility versus blend-in trade.

An encrypted, authenticated last mile only protects the client-to-resolver hop. It does nothing for the resolver-to-authoritative path, which is still plain port 53 upstream (DNSSEC covers integrity there, not confidentiality). It does not hide which resolver you use, and it does not stop a malicious or misconfigured resolver from lying to you. Only client-side DNSSEC validation (Phase 6, Client B) catches that.

Phases 4, 5, 6 and the Phase 7 attack half (Sabil)

- **Phase 4, signature stripping and downgrade.** Extend `forge_or_relay()` in `common/interceptor/dns_intercept.` (the `PHASE4` hook) with a strip mode.
- **Phase 5, zone walking.** Walk the Phase 3 NSEC zone, then fix it by re-signing with `NSEC3=1 sh run.sh` in `common/signing`.
- **Phase 6, last-mile forgery.** Same tool, diversion on edge-router, victim is the client (a plain stub vs a validating one).
- **Phase 7 attack half.** The three attempts against the two client types.

Appendix: Generative AI usage

We used AI assistance (Claude Code) to build this. What it did and how we checked it:

- **What it did:** scaffolded the repo; wrote the Compose files, BIND zone and daemon configs, and the routing scripts; wrote the DNSSEC signing scripts; wrote the C/libpcap interceptor; wrote the Phase 7 PKI and DoT/DoH config; and drafted this report from the captured output.
- **Example prompts:** "split this 7-phase DNS assignment evenly"; "build the Phase 1 base: compose, zones, routing"; "write the DNSSEC signing scripts and a validating resolver"; "write a C/libpcap on-path interceptor with a selective A-record forge"; "stand up the Phase 7 CA plus DoT/DoH and check there is no cleartext".
- **How we verified it:** we did not take any of it on trust. Every phase was deployed on Docker and checked against the brief's own success criteria. Phase 1: resolution, fetch, isolation, and a referral capture. Phase 2: relay is transparent, then forge sends example.com to the rogue while www is untouched. Phase 3: the AD flag and `delv` saying "fully validated", then the re-run attack giving SERVFAIL with the matching log line. Phase 7: DoT and DoH resolution, two deliberate negative certificate tests that both fail, and a tcpdump on edge-router showing zero cleartext on port 53. All commands are in each phase's `scripts/`.